CODEX TÉCNICO (PDF MESTRE)

Gestão Financeira Pessoal • Blueprint + Base de Conhecimento

Consolidação v1 → v8 (Fevereiro/2026).

Como usar este material
• Use os capítulos 1–12 para modelagem de domínio (produto) e regras. • Use os capítulos 13–14 para integrações e segurança (base para agentes). • Use os capítulos 15–16 para operar o produto em produção (SLO, incidentes, evals). • Use o Apêndice C como referência de schemas/contratos e para manter auditabilidade.

Índice Geral (v1 → v8)

Dimensões de domínio (Produto)

- 1. Fundamentos: objetivos, diagnóstico e modelo mental
- 2. Orçamento & fluxo de caixa
- 3. Metas & planejamento (buckets)
- 4. Patrimônio & balanço pessoal
- 5. Finanças comportamentais
- 6. Dívidas
- 7. Investimentos: Renda Fixa
- 8. Investimentos: Renda Variável
- 9. Carteira & rebalanceamento
- 10. Aposentadoria / Independência Financeira
- 11. Impostos & otimização fiscal
- 12. Seguros & proteção patrimonial

Plataforma (Dados + Segurança + Operação + Agentes)

- 13. Open Finance & integrações / pipelines
- 14. Privacidade, consentimento, segurança e auditoria
- 15. Operações do produto: métricas, qualidade, incidentes, SLOs
- 16. Plataforma de agentes: roteamento, ferramentas, memória, avaliação, red-teaming

Apêndice C

- Schemas de Alert/Consent/AgentAction + DSL de regras
- Schemas de carteira (Position/PortfolioTarget) + funções de janela da DSL
- Schemas de preço/performance/câmbio (PriceSnapshot/Performance/FxRate)
- Schemas de impostos/seguros (TaxEvent/Policy/Claim)
- Schemas de integrações e segurança (DataConnection/IngestionJob/AuditLog/Permission)
- Schemas operacionais (Incident/SLO/EvalRun/PromptPackage)

CODEX TÉCNICO

Gestão Financeira Pessoal

Blueprint completo para construção de sistemas, agentes autônomos e produtos financeiros orientados por dados, regras e comportamento humano.

O que este documento entrega
Uma base de conhecimento estruturada em 17 dimensões, com: ontologia (modelo do mundo), padrões de dados e cálculo, motor de regras, arquitetura de agentes, padrões de UX e templates reutilizáveis.

Escopo: Brasil como referência principal. Conceitos são universais; parâmetros (tributos, produtos, regras) devem ser ajustáveis.

Leitor-alvo: product builders, engenheiros, analistas, designers e autores de agentes.

Índice

- 1. Como usar este Codex (fluxo de trabalho)
- 2. Arquitetura de conhecimento (camadas e módulos)
- 3. Ontologia e modelo de dados (conceitual)
- 4. Motor de regras (padrão + execução + auditabilidade)
- 5. Arquitetura de agentes autônomos (especialistas + orquestrador + guardrails)
- 6. Template oficial de capítulo (o molde das 17 dimensões)
- 7. Capítulo 1 - Orçamento e Fluxo de Caixa
- 8. Capítulo 2 - Reserva de Emergência
- 9. Apêndices (KPIs, schemas JSON, mensagens, testes) - estrutura v1

Nota de versionamento
Este é o v1 com 2 capítulos completos (Dimensões 1 e 2) para validar formato e profundidade. Os demais capítulos seguem o mesmo molde.

1. Como usar este Codex

Use este documento como fonte única de verdade para três frentes: (a) produto e UX; (b) engenharia e dados; (c) agentes autônomos.

Fluxo recomendado (do paper ao código):

- Escolha uma dimensão e implemente o mínimo do capítulo (dados -> KPIs -> regras -> UX -> testes).
- Instrumente auditoria: toda recomendação precisa registrar evidência e regra disparada.
- Progrida por camadas: primeiro determinístico (KPIs/regras), depois inteligência (priorização e narrativa).
- Evite big bang: a robustez nasce de loops curtos (importar -> calcular -> explicar -> agir).

Definição operacional de 'poderoso'
Poderoso significa: explicável, testável, versionável e reaproveitável. Se não dá para testar e explicar, não é conhecimento - é opinião.

2. Arquitetura de conhecimento

Organize o repositório (e a cabeça do sistema) em camadas. Cada camada tem artefatos próprios.

Camada	Responsabilidade	Artefatos (o que versionar)
Dados	Verdade e rastreabilidade	Schemas, normalização, dedup, contratos de importação
Cálculos	KPIs e projeções determinísticas	Fórmulas, testes unitários, parâmetros (inflacao, IR)
Regras	Diagnóstico acionável	Catálogo de regras, logs de execução, severidade, evidência
Agentes	Planejamento e priorização	Prompts, ferramentas, guardrails, orquestração
UX	Decisão em 1 clique	Telas, microcopy, estados, alertas e fricções

Cada dimensão (1..17) é um módulo. Os módulos compartilham ontologia e padrões, mas têm regras e UX específicas.

3. Ontologia e modelo de dados (conceitual)

A ontologia é o dicionário do mundo. Ela garante consistência entre produto, regras e agentes.

Entidades mínimas

- Account (conta): onde o dinheiro vive.
- Transaction (realizado) e PlannedTransaction (previsto).
- Category: classificação (income/expense/transfer).
- Budget: teto por categoria e período.
- Goal: meta com valor, prazo, prioridade e horizonte.
- Fund/Bucket: caixas mentais (emergência, curto, médio, longo).
- Debt e Asset: passivos e ativos (inclui real: imóvel/carro).
- Alert e Recommendation: o que o sistema diz e por quê.
- RuleExecution e AgentAction: auditabilidade do raciocínio.
- Consent: trilha de permissões e privacidade.

Padrão de auditabilidade
Toda Recommendation deve apontar (1) regra disparada; (2) evidência (KPIs + transações agregadas); (3) ação sugerida; (4) risco e trade-off; (5) estado (open/done/snoozed).

4. Motor de regras

O motor de regras transforma dados em diagnósticos acionáveis. Ele deve ser determinístico, rastreável e testável.

Forma padrão de uma regra

- Input: conjuntos de dados e KPIs necessários.
- Pré-condições: quando a regra faz sentido (ex.: dados completos).
- Condição: expressão booleana (ex.: reserva_meses < 3).
- Severidade: 1..5 (impacto e urgência).
- Saída: Alert/Recommendation + ActionPlan.
- Evidência: números e agregações usadas.
- Mensagem: explicação curta + por que importa.
- Guardrails: quando NÃO recomendar (ex.: dados insuficientes).

Exemplo de pseudo-schema (human readable)

```
rule_id: RESERVE_MIN_3M
when: MonthClosed OR TransactionCreated
if: reserve_months < 3 AND essential_cost_monthly > 0
then: create_recommendation(severity=5, action='build_reserve', evidence=[...])
```

5. Arquitetura de agentes autônomos

Agentes são especialistas. O orquestrador junta as peças, resolve conflitos e entrega um plano curto.

Agentes mínimos (v1)

- AgentBudget: orçamento, tetos, vazamentos.
- AgentReserve: reserva, recomposição, regra de uso.
- AgentGoals: metas e aporte necessário.
- AgentDebt: quitação e prevenção.
- AgentOrchestrator: gera plano semanal/mensal com no máximo 7 ações.

Guardrails (não negociável)
Agentes nunca executam transações. Eles propõem. Toda proposta deve: (1) declarar suposições; (2) mostrar impacto em KPI; (3) oferecer alternativa conservadora; (4) exigir confirmação explícita para qualquer ação irreversível.

6. Template oficial de capítulo

Este molde é repetido nas 17 dimensões para garantir consistência e reuso.

- 6.1 Definições e escopo (o que entra, o que não entra)
- 6.2 Objetivo do capítulo (o problema que resolve)
- 6.3 Decisões-chave (2-5 escolhas que definem o jogo)
- 6.4 Etapas (passo a passo humano + passo a passo do sistema)
- 6.5 KPIs e fórmulas (com interpretação e limites)
- 6.6 Catálogo de regras (gatilhos, severidade, evidência, outputs)
- 6.7 Padrões de UX (telas, estados, microcopy, fricções)
- 6.8 Agente do capítulo (missão, ferramentas, guardrails, outputs)
- 6.9 Testes (unit, integration, E2E) + cenários de borda
- 6.10 Checklist de implementação (Definition of Done)

7. Capítulo 1 - Orçamento e Fluxo de Caixa

7.1 Definições e escopo

Orçamento é a decisão prévia de como alocar recursos no tempo. Fluxo de caixa é a execução (realizado) e a previsão (previsto) dessa alocação. Este capítulo cobre receitas, despesas, transferências, recorrências e provisionamentos. Não cobre ainda investimentos (capítulos 7-9) nem impostos (capítulo 14).

7.2 Objetivo

- Criar previsibilidade mensal.
- Evitar estourar categorias críticas.
- Garantir aporte para 'Futuro' (reserva/investimento/metas).

7.3 Decisões-chave

- Método: 50/30/20, base-zero, ou teto por categoria (recomendado: teto por categoria + provisionamento).
- Granularidade de categorias (poucas para adesão; suficientes para diagnóstico).
- Tratamento de despesas grandes/anuais (provisionar ou sofrer).
- Regra de 'gasto variável' (limites e fricções).

7.4 Etapas - humano

- Diagnóstico: listar renda líquida e custos essenciais.
- Categorizar: 100% das transações com categoria e tipo (fixo/variável).
- Definir tetos: por categoria e por mês (com alertas 80/100/120).
- Provisionar: transformar anuais em parcelas mensais.
- Fechar mês: previsto vs realizado + ajuste do mês seguinte.

7.5 Etapas - sistema

- Ingestão de transações (manual/import/Open Finance) + deduplicação.
- Detecção de recorrências (assinaturas, salários, contas).
- Sugestão de categorias (com confiança) + confirmação rápida.
- Cálculo de tetos e alertas em tempo real.
- Fechamento mensal automático + recomendações priorizadas.

7.6 KPIs e fórmulas

KPI	Fórmula	Interpretação
Taxa de Poupança	(Aporte Futuro) / (Renda líquida)	Quanto do mês vai para construção patrimonial
Desvio Orçamentário	abs(real - previsto) / previsto	Aderência ao plano; alto desvio indica falta de provisão ou tetos irr
% Gasto Fixo	Fixos / Renda líquida	Pressão estrutural; quanto maior, menor margem de manobra
Runway do Mês	Saldo projetado fim do mês	Se o mês fecha negativo, ajuste imediato de tetos/recorrências

7.7 Catálogo de regras (v1)

- BUDGET_80: quando categoria atingir 80% do teto -> alerta 'atenção' + sugestão de corte.
- BUDGET_100: ao atingir 100% -> alerta 'crítico' + bloquear recomendações de novos gastos (fricção).
- RECURRENCE_NEW: detectar recorrência não marcada -> sugerir 'fixar' e encaixar no orçamento.
- PROVISION_MISSING: identificar despesa anual grande -> sugerir provisionamento mensal.
- SAVINGS_TOO_LOW: taxa de poupança < X por 3 meses -> recomendação de replanejamento de tetos.

7.8 UX patterns

- Tela 'Hoje': saldo + 3 alertas no máximo (o resto fica em fila).
- Tela 'Categorias': barra de progresso (80/100/120) com ações rápidas.
- Tela 'Fechamento do mês': 3 desvios principais + 3 ações para o próximo mês.
- Fricção: regra 24h para compra acima de um limite (configurável).

7.9 Testes essenciais

- Unit: cálculo de desvio, taxa de poupança, projeção de saldo do mês.
- Integration: deduplicação de importação + pareamento de transferências.
- E2E: usuário estoura uma categoria -> recebe alerta correto -> ajusta teto -> fechamento mensal reflete ajuste.

Definition of Done - Capítulo 1
1) 95% das transações categorizadas; 2) tetos ativos; 3) alertas 80/100/120 funcionando; 4) fechamento mensal reproduzível; 5) logs de regras e recomendações auditáveis; 6) testes unit + E2E passando.

8. Capítulo 2 - Reserva de Emergência

8.1 Definições e escopo

Reserva de emergência é um montante líquido para absorver choques (perda de renda, despesa médica, evento inesperado). É diferente de: investimento de longo prazo, oportunidade, viagem e consumo. Deve ter liquidez, baixo risco e regra de uso clara.

8.2 Objetivo

- Evitar dívida cara em crises.
- Impedir venda de ativos em momento ruim.
- Aumentar estabilidade emocional para manter estratégia.

8.3 Decisões-chave

- Tamanho (meses de cobertura) conforme estabilidade da renda e dependentes.
- Onde guardar (liquidez e risco).
- O que conta como emergência (política objetiva).
- Regras de recomposição após uso.

8.4 Etapas - humano

- Calcular custo essencial mensal (moradia, alimentação, saúde, educação, transporte essencial).
- Definir alvo (3-12 meses; adicionar margem para dependentes).
- Construir com aporte automático até completar.
- Política de uso: critérios claros + registro do motivo.
- Recompor: quando usar, suspender aportes de risco e recompor primeiro.

8.5 Etapas - sistema

- Estimar custo essencial via categorias marcadas como 'essenciais' (com ajuste manual).
- Calcular meses de cobertura (reserve_balance / essential_cost_monthly).
- Sugerir meta-alvo baseada em perfil (CLT, variável, empresário) e dependentes (como parâmetros).
- Criar Bucket 'Emergência' e automatizar aporte (quando integrado a bancos).
- Detectar saques do bucket e disparar plano de recomposição.

8.6 KPIs e fórmulas

KPI	Fórmula	Interpretação
Meses de Reserva	Saldo Reserva / Custo essencial mensal	Cobertura de choque; alvo típico 3-12
Tempo até Meta	(Meta - saldo) / aporte_mensal	Velocidade de construção
Taxa de Recomposição	aporte_recomposicao / aporte_total	Prioridade correta após uso

8.7 Catálogo de regras (v1)

- RESERVE_MIN_3M: meses_reserva < 3 -> recomendação crítica: construir reserva imediatamente.

- RESERVE_TARGET_BY_PROFILE: calcular alvo com base em estabilidade de renda e dependentes (parâmetros).
- RESERVE_WITHDRAWAL: saque do bucket -> solicitar motivo + disparar plano de recomposição.
- RESERVE_EXCESS: meses_reserva > 12 (ou alvo + margem) -> sugerir realocar excedente para metas/investimentos.

8.8 UX patterns

- Widget: 'Meses de Reserva' com meta e projeção de conclusão.
- Botão: 'Marcar como emergência' (obriga motivo).
- Modo crise: se renda cair, o app prioriza ações de proteção e reduz ruído.

8.9 Testes essenciais

- Unit: cálculo de meses_reserva e tempo_ate_meta.
- Integration: bucket separado não mistura com investimentos.
- E2E: usuário saca reserva -> app registra motivo -> gera plano de recomposição -> atualiza KPIs.

Definition of Done - Capítulo 2
1) custo essencial configurável e auditável; 2) meses de reserva calculado; 3) regras RESERVE_MIN e WITHDRAWAL ativas; 4) UI com meta e projeção; 5) trilha de motivo para saque; 6) testes unit + E2E passando.

9. Apêndices (estrutura v1)

Os apêndices tornam o Codex executável: fórmulas, schemas, mensagens e testes. Nesta versão, deixamos a estrutura pronta.

- A. Catálogo de KPIs: definição, fórmula, interpretação, limites, unidades.
- B. Catálogo de Regras: id, condição, severidade, mensagem, evidência mínima, ações.
- C. Schemas JSON: rule, recommendation, alert, audit log.
- D. Biblioteca de Mensagens: microcopy por estado (info/atenção/crítico) e por dimensão.
- E. QA: casos E2E + gerador de dados + agente 'usuário robô'.
- F. Segurança e Privacidade: consentimentos, retenção, exportação, logs.

Próxima expansão
A partir deste molde validado, os capítulos 3-17 serão preenchidos no mesmo padrão. Em paralelo, vamos consolidar os apêndices B e C para permitir um motor de regras versionável e testável.

CODEX TÉCNICO

Gestão Financeira Pessoal

Blueprint completo para construção de sistemas, agentes autônomos e produtos. v2 adiciona Dimensões 3 e 4 + início do Apêndice C (Schemas JSON).

Novidades desta versão (v2)
• Capítulo 3 completo (Planejamento/Metas) • Capítulo 4 completo (Educação Financeira aplicada) • Apêndice C iniciado com schemas JSON base para motor de regras e auditoria.

Índice

- 1. Como usar este Codex (fluxo de trabalho)
- 2. Arquitetura de conhecimento (camadas e módulos)
- 3. Ontologia e modelo de dados (conceitual)
- 4. Motor de regras (padrão + execução + auditabilidade)
- 5. Arquitetura de agentes autônomos (especialistas + orquestrador + guardrails)
- 6. Template oficial de capítulo
- 7. Capítulo 1 - Orçamento e Fluxo de Caixa
- 8. Capítulo 2 - Reserva de Emergência
- 9. Capítulo 3 - Planejamento Financeiro e Metas
- 10. Capítulo 4 - Educação Financeira aplicada (dentro do produto)
- 11. Apêndices (estrutura) + Apêndice C (Schemas JSON v1)

1. Como usar este Codex

Aplique por módulos: implemente 1 dimensão por vez (dados → KPIs → regras → UX → testes). Cada recomendação deve ser explicável e auditável.

- Loop curto: importar/cadastrar → calcular → recomendar → agir → medir.
- Primeiro determinístico: KPIs e regras simples antes de qualquer IA mais sofisticada.
- Auditoria sempre: Recommendation aponta regra + evidência + trade-offs.

2. Arquitetura de conhecimento

Camada	Responsabilidade	Artefatos (versionar)
Dados	Verdade e rastreabilidade	Schemas, normalização, dedup, contratos de importação
Cálculos	KPIs e projeções	Fórmulas, parâmetros, testes unitários
Regras	Diagnóstico acionável	Catálogo de regras, severidade, evidência mínima, logs
Agentes	Planejamento e priorização	Prompts, ferramentas, guardrails, orquestração
UX	Decisão	Telas, estados, microcopy, fricções e alertas

3. Ontologia e modelo de dados (conceitual)

- Account, Transaction, PlannedTransaction, Category, Budget
- Goal (meta), Fund/Bucket (caixa), Debt, Asset, Portfolio
- Alert, Recommendation, RuleExecution, AgentAction, Consent

Princípio
Tudo que o sistema diz vira um objeto versionável e auditável (Recommendation + RuleExecution).

4. Motor de regras

- Regras são determinísticas por padrão, com parâmetros configuráveis.
- Toda regra tem: input → pré-condições → condição → severidade → saída → evidência → mensagem → guardrails.
- Execução por eventos (TransactionCreated, MonthClosed, IncomeChanged, etc.).

Pseudo-schema (human readable):

```
rule_id: GOAL_UNDERFUNDED
when: MonthClosed
if: goal_progress_pct < expected_progress_pct - tolerance
then: create_recommendation(action='increase_contribution', severity=3,
evidence=[...])
```

5. Arquitetura de agentes

- Agentes especialistas por dimensão geram diagnósticos e planos (não executam dinheiro).
- Orquestrador limita a saída a 3–7 ações priorizadas por impacto x esforço x risco.
- Guardrails: suposições explícitas + alternativa conservadora + confirmação para ações irreversíveis.

6. Template oficial de capítulo

- Definições e escopo
- Objetivo
- Decisões-chave
- Etapas (humano + sistema)
- KPIs e fórmulas
- Regras (gatilhos e outputs)
- UX patterns
- Agente do capítulo
- Testes + bordas
- Checklist (DoD)

7. Capítulo 1 - Orçamento e Fluxo de Caixa (resumo operacional)

Nesta versão, o conteúdo completo do Capítulo 1 permanece válido. Aqui fica o resumo para navegação rápida.

- Etapas: ingestão → categorização → tetos → alertas 80/100/120 → fechamento mensal.
- KPIs: taxa de poupança, desvio orçamentário, % gasto fixo, runway do mês.
- Regras v1: BUDGET_80/100, RECURRENCE_NEW, PROVISION_MISSING, SAVINGS_TOO_LOW.
- DoD: fechamento mensal reproduzível + logs + testes unit e E2E.

8. Capítulo 2 - Reserva de Emergência (resumo operacional)

- Etapas: custo essencial → meses-alvo → bucket emergência → aporte automático → política de uso → recomposição.
- KPIs: meses de reserva, tempo até meta, taxa de recomposição.
- Regras v1: RESERVE_MIN_3M, RESERVE_TARGET_BY_PROFILE, RESERVE_WITHDRAWAL, RESERVE_EXCESS.
- DoD: cálculo auditável + motivo de saque + plano de recomposição + testes.

9. Capítulo 3 - Planejamento Financeiro e Metas

9.1 Definições e escopo

Planejamento financeiro aqui significa traduzir objetivos em números e rotinas: valor-alvo, prazo, inflação, aporte necessário e rastreio de progresso. Este capítulo cobre metas (curto/médio/longo), buckets, priorização e revisão. Não cobre ainda alocação detalhada por ativos (capítulos 7-10).

9.2 Objetivo

- Converter desejos em metas executáveis (valor, prazo, aporte).
- Separar objetivos por horizonte para evitar mistura de risco e liquidez.
- Manter o usuário em progresso contínuo com revisões e alertas de desvio.

9.3 Decisões-chave

- Horizontes: curto (≤2 anos), médio (3–10), longo (10+).
- Prioridade: essencial/estratégico/desejo (ou 1..5).
- Inflação: parâmetro ajustável (ex.: IPCA esperado) por horizonte.
- Método de aporte: fixo mensal vs variável (percentual da renda).
- Critério de sucesso: valor nominal, valor real (recomendado: real).

9.4 Etapas - humano

- Lista de metas: 3–10 metas no máximo (evitar dispersão).
- Parametrizar: valor-alvo, data alvo, prioridade, moeda, tipo (compra/educação/aposentadoria).
- Escolher bucket: emergência / curto / médio / longo.
- Definir aporte: quanto por mês (ou % da renda).
- Revisar: trimestral (ajustes leves) e anual (replanejamento).

9.5 Etapas - sistema

- Assistente de criação de meta (wizard) com padrões por tipo de objetivo.
- Cálculo automático de aporte necessário (com premissas explícitas).
- Tracking de progresso: saldo/valor-alvo, % concluído, atraso/adiantamento.
- Rebalanceamento entre metas (se uma está atrasada e outra adiantada).
- Alertas: meta subfinanciada, prazo apertando, mudança de renda.

9.6 KPIs e fórmulas

KPI	Fórmula (simplificada)	Interpretação
Progresso da Meta (%)	saldo_meta / valor_alvo	Quanto já foi financiado
Progresso Esperado (%)	meses_passados / meses_totais	Ritmo esperado linear (baseline)
Gap de Aporte	aporte_necessario - aporte_atual	Quanto falta para voltar ao ritmo
Atraso (meses)	meses_para_meta_com_aporte_atual - meses_restantes	Se > 0, sistema está atrasada

9.7 Catálogo de regras (v1)

- GOAL_UNDERFUNDED: progresso_atual < progresso_esperado - tolerância → recomendar aumentar aporte ou ajustar prazo.
- GOAL_DEADLINE_CLOSE: meses_restantes < limiar e gap_aporte > 0 → recomendação de replanejamento imediato.
- GOAL_TOO_MANY: metas_ativas > 10 → sugerir consolidar e priorizar (reduz dispersão).
- GOAL_BUCKET_MISMATCH: meta curta alocada em bucket longo (risco) → alertar inconsistência de horizonte.
- INCOME_CHANGED_REPLAN: variação de renda > X% → recalcular aportes e sugerir redistribuição.

9.8 UX patterns

- Tela 'Metas': cards com progresso, ritmo e gap (1 ação por card).
- Wizard: meta → horizonte → bucket → aporte → confirmação (com premissas).
- Fechamento mensal: 1 bloco 'Metas' com 2–3 ajustes recomendados.

9.9 Agente do capítulo (AgentGoals)

- Missão: manter metas no trilho com o mínimo de ações.
- Entradas: lista de metas, saldos, aportes, horizonte, renda e mudanças recentes.
- Saídas: plano de 3 ações (aumentar aporte, ajustar prazo, reduzir metas) com impacto em KPIs.
- Guardrails: nunca sugerir risco alto para metas curtas; sempre declarar premissas de inflação/retorno.

9.10 Testes essenciais

- Unit: cálculo de progresso_esperado, gap_aporte, atraso_meses.
- Integration: meta vinculada a bucket e transações planejadas/realizadas atualiza saldo corretamente.
- E2E: renda cai 20% → metas recalculadas → orquestrador prioriza preservação (emergência + essenciais).

Definition of Done - Capítulo 3
1) metas com horizonte/bucket; 2) KPIs de progresso e atraso; 3) regras GOAL_* ativas e auditáveis; 4) UI de metas com ação única; 5) integração com fechamento mensal; 6) testes unit+E2E.

10. Capítulo 4 - Educação Financeira aplicada

10.1 Definições e escopo

Educação financeira, aqui, é o conjunto de explicações e simuladores que aumentam a qualidade de decisão do usuário usando os próprios dados dele. Não é curso. É aprendizagem contextual: no momento da decisão.

10.2 Objetivo

- Reduzir erros previsíveis (ex.: confundir retorno nominal com real).
- Aumentar autonomia do usuário ao escolher prazos, liquidez e risco.
- Criar linguagem comum para recomendações e agentes (sem jargão).

10.3 Decisões-chave

- Nível de profundidade (básico, intermediário, avançado) configurável.
- Formato: tooltips curtos + simuladores simples (preferível) vs textos longos.
- Medição de aprendizado: entender sem fricção (não virar 'aula').

10.4 Etapas - humano

- Aprender com o que já aconteceu: fechar mês e ver onde errou (e por quê).
- Tomar 1 decisão melhor por semana (ex.: provisionar anual, reduzir assinatura inútil).
- Revisar conceitos só quando forem úteis para uma decisão real.

10.5 Etapas - sistema

- Explicações sempre ligadas a um KPI ou ação: 'por que isso importa agora?'.
- Simuladores embutidos: inflação (nominal vs real), imposto (bruto vs líquido), tempo (juros compostos).
- Microcopy em 3 níveis: info → atenção → crítico (sem moralismo).
- Registro opcional de 'aprendizados' (para reforço e personalização).

10.6 KPIs e métricas de produto

Métrica	Como medir	Por que importa
Taxa de Ação	% recomendações concluídas	Educação só funciona se vira ação
Erros evitáveis	queda em estourar teto / usar rotativo / resgatar melhor	Mostra melhora real
Tempo por revisão	minutos na revisão semanal/mensal	Se for alto demais, o sistema não escala
Compreensão	cliques em 'entendi' / uso de simulador	Sinal indireto de aprendizado

10.7 Catálogo de regras (v1)

- EDU_NOMINAL_VS_REAL: sempre que mostrar retorno, exibir também retorno real estimado (com premissa de inflação).
- EDU_BRUTO_VS_LIQUIDO: ao comparar produtos/retornos, exibir imposto e custo como linha separada.
- EDU_LIQUIDEZ_MISMATCH: meta curta com ativo ilíquido → explicar risco de marcação a mercado/liquidez.
- EDU_COST_OPPORTUNITY: compra grande (capex) → mostrar custo de oportunidade (simulação simples).

10.8 UX patterns e microcopy

- Tooltip curto (máx. 2 linhas) + botão 'simular'.
- Simulador: 3 inputs no máximo (valor, tempo, taxa) + resultado em 1 gráfico/linha.
- Padrão de linguagem: fato → impacto → ação. Ex.: 'Seu retorno foi 12% (bruto). Com inflação 8%, seu ganho real foi ~4%. Quer ver como isso muda sua meta?'

10.9 Agente do capítulo (AgentEducation)

- Missão: traduzir termos em decisões simples no contexto do usuário.
- Saída: explicação curta + simulador recomendado + ação prática.
- Guardrails: evitar jargão; declarar premissas; não prometer retorno; não induzir risco.

10.10 Testes essenciais

- Unit: cálculo nominal→real e bruto→líquido com parâmetros.
- Integration: componentes de tooltip/simulador aparecem apenas quando relevantes (não poluir UI).
- E2E: usuário compara dois investimentos → UI mostra bruto/líquido/real consistentemente.

Definition of Done - Capítulo 4
1) microcopy fato→impacto→ação; 2) simuladores mínimos (inflacao, imposto, juros); 3) regras EDU_* ativas; 4) métricas de produto instrumentadas; 5) testes unit+E2E.

11. Apêndices (estrutura) + Apêndice C (Schemas JSON v1)

Nesta versão iniciamos o Apêndice C com schemas JSON base. Eles permitem versionar regras, recomendações e auditoria.

Apêndice C.1 - rule.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/rule.schema.json", "title": "Rule", "type": "object",
"required": ["rule_id", "dimension_id", "version", "when", "inputs", "condition",
"severity", "output"], "properties": { "rule_id": {"type": "string"},
"dimension_id": {"type": "integer", "minimum": 1, "maximum": 17}, "version":
{"type": "string"}, "when": {"type": "array", "items": {"type": "string"}},
"inputs": {"type": "array", "items": {"type": "string"}}, "preconditions":
{"type": "array", "items": {"type": "string"}}, "condition": {"type": "string",
"description": "Boolean expression evaluated by the rule engine"}, "severity":
{"type": "integer", "minimum": 1, "maximum": 5}, "output": { "type": "object",
"required": ["action_type", "title_template", "message_template", "evidence_min"],
"properties": { "action_type": {"type": "string"}, "title_template": {"type":
"string"}, "message_template": {"type": "string"}, "evidence_min": {"type":
"array", "items": {"type": "string"}}, "risk_level": {"type": "string", "enum":
["low", "medium", "high"]}, "requires_consent": {"type": "boolean"} } },
"guardrails": {"type": "array", "items": {"type": "string"}}, "params": {"type":
"object", "additionalProperties": true}, "tags": {"type": "array", "items":
{"type": "string"}} } }
```

Apêndice C.2 - recommendation.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/recommendation.schema.json", "title":
"Recommendation", "type": "object", "required": ["id", "user_id", "dimension_id",
"rule_id", "severity", "title", "rationale", "action_type", "evidence"],
"properties": { "id": {"type": "string"}, "user_id": {"type": "string"},
"dimension_id": {"type": "integer"}, "rule_id": {"type": "string"}, "severity":
{"type": "integer", "minimum": 1, "maximum": 5}, "confidence": {"type": "number",
"minimum": 0, "maximum": 1}, "title": {"type": "string"}, "rationale": {"type":
"string"}, "action_type": {"type": "string"}, "action_plan": {"type": "array",
"items": {"type": "string"}}, "tradeoffs": {"type": "array", "items": {"type":
"string"}}, "assumptions": {"type": "array", "items": {"type": "string"}},
"evidence": {"type": "object", "additionalProperties": true}, "status": {"type":
"string", "enum": ["open", "done", "snoozed"]}, "created_at": {"type": "string",
"format": "date-time"} } }
```

Apêndice C.3 - rule_execution.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/rule_execution.schema.json", "title":
"RuleExecution", "type": "object", "required": ["id", "user_id", "rule_id",
"evaluated_at", "result", "evidence"], "properties": { "id": {"type": "string"},
"user_id": {"type": "string"}, "rule_id": {"type": "string"}, "evaluated_at":
{"type": "string", "format": "date-time"}, "result": {"type": "string", "enum":
["fired", "not_fired", "skipped"]}, "skip_reason": {"type": "string"}, "evidence":
{"type": "object", "additionalProperties": true}, "engine_version": {"type":
"string"} } }
```

Nota

Os schemas acima são base. Em versões futuras adicionaremos: Alert schema, Consent schema, AgentAction schema, e um DSL para condições (evitar strings livres).

CODEX TÉCNICO

Gestão Financeira Pessoal

v3 adiciona Dimensões 5 e 6 + evolução do Apêndice C (schemas adicionais) + proposta de DSL para condições.

Novidades desta versão (v3)
• Capítulo 5 completo (Finanças Comportamentais) • Capítulo 6 completo (Gestão de Dívidas) • Apêndice C: Alert/Consent/AgentAction + DSL simples para conditions.

Índice (v3)

- Capítulo 5 - Finanças Comportamentais
- Capítulo 6 - Gestão de Dívidas
- Apêndice C (continuação): schemas JSON (Alert, Consent, AgentAction) + DSL de condições

Capítulo 5 - Finanças Comportamentais

5.1 Definições e escopo

Finanças comportamentais, no produto, é a camada que detecta padrões de decisão (gatilhos, impulsos, aversão à perda, excesso de confiança) e aplica intervenções mínimas para reduzir erro previsível. Não é terapia; é desenho de ambiente e feedback.

5.2 Objetivo

- Reduzir decisões impulsivas (gasto e investimento).
- Diminuir oscilação emocional que quebra o plano (venda em queda, resgate por ansiedade).
- Aumentar consistência: menos força de vontade, mais sistema.

5.3 Decisões-chave

- Quais comportamentos serão monitorados (gasto, resgates, giro de carteira, rotativo).
- Quais intervenções serão usadas: fricção, limites, pausas, alternativas.
- Nível de intrusão (configurável): suave / padrão / rígido.
- Como medir melhoria: queda de eventos ruins + aumento de ações concluídas.

5.4 Etapas - humano

- Reconhecer gatilhos (datas, contextos, emoções) que antecedem gastos/decisões ruins.
- Criar regras pessoais simples (ex.: 24h para compra acima de X).
- Revisar semanalmente 10 minutos: 'o que me puxou para fora do plano?'.

5.5 Etapas - sistema

- Detectar padrões: picos pós-salário, fim de semana, madrugada, 'microcompras'.
- Detectar 'eventos críticos': estouro de teto, uso de rotativo, saque de reserva, venda em queda.
- Aplicar intervenção mínima (não moralista): pausa + pergunta + alternativa.
- Aprender com feedback do usuário (marcar: necessário/impulso/erro).

5.6 KPIs e métricas

KPI/Métrica	Definição	Uso
Índice de Impulso (gastos)	% transações 'não planejadas' acima de X	Sinaliza fragilidade do orçamento
Eventos Críticos	contagem: rotativo/saque reserva/estouro teto	Alvos de intervenção prioritários
Giro de Carteira	vendas+compras / patrimônio investido	Detecta trading emocional
Taxa de Ação	% recomendações concluídas	Mede se feedback virou execução

5.7 Catálogo de regras (v1)

- BEH_24H_COOLDOWN: compra acima de limite → sugerir regra 24h + registrar decisão depois.

- BEH_POST_SALARY_SPIKE: pico de gasto 48h após salário → alerta e ajuste de tetos/limites.
- BEH_NIGHT_SPEND: transações após 23h recorrentes → sugerir fricção e revisão de gatilhos.
- BEH_PANIC_SELL: venda após queda relevante (se integrar preços) → explicar risco de sequência e sugerir pausa.
- BEH_RECURRING_LEAK: assinaturas raramente usadas → sugestão de cancelamento com economia projetada.

5.8 UX patterns

- Fricção leve: confirmação com 2 botões ("necessário" / "posso esperar").
- Tela 'Padrões': 3 insights máximos por semana (evitar overload).
- Mensagens sem julgamento: fato → impacto → alternativa.
- Configuração de limites pessoais por categoria e por evento (rotativo/resgate).

5.9 Agente do capítulo (AgentBehavior)

- Missão: reduzir eventos críticos com intervenções pequenas.
- Saída: 1) padrões detectados; 2) 1 intervenção sugerida; 3) 1 métrica para acompanhar.
- Guardrails: evitar moralismo; não inferir saúde mental; declarar incerteza (padrões são probabilísticos).

5.10 Testes essenciais

- Unit: detecção de spikes por janela móvel; regras de horário; classificação planejado vs não planejado.
- Integration: eventos críticos disparam alertas e entram na fila do orquestrador.
- E2E: compra acima do limite → fricção 24h → usuário decide → sistema registra e ajusta risco/limites.

Definition of Done - Capítulo 5
1) detecção de pelo menos 3 padrões; 2) fricções configuráveis; 3) eventos críticos instrumentados; 4) recomendações com economia/impacto; 5) testes unit+E2E.

Capítulo 6 - Gestão de Dívidas

6.1 Definições e escopo

Dívida é uma obrigação futura com custo (juros) e restrições (prazo, garantias). Gestão de dívidas no sistema cobre inventário, custo efetivo (CET quando disponível), priorização de pagamento, simulações e prevenção (evitar rotativos).

6.2 Objetivo

- Reduzir custo total de juros (dinheiro queimado).
- Diminuir risco de insolvência (atrasos e bola de neve).
- Criar plano executável com marcos mensais.

6.3 Decisões-chave

- Método: avalanche (maior juros primeiro) vs bola de neve (menor saldo primeiro).
- Critério de prioridade: taxa efetiva, risco de atraso, garantia, impacto emocional.
- Regra de alocação de excedente: todo extra vai para a dívida prioritária até quitar.
- Como tratar refinanciamento/renegociação (simular antes).

6.4 Etapas - humano

- Inventariar todas as dívidas: saldo, parcela, taxa, prazo, credor, garantia.
- Separar dívidas tóxicas (rotativo, cheque especial) das estruturadas (imobiliário).
- Escolher método (avalanche recomendado financeiramente; snowball pode ajudar execução).
- Criar plano mensal e acompanhar redução do saldo.
- Prevenir recaída: teto de cartão, reserva mínima, regra de não-rolagem.

6.5 Etapas - sistema

- Cadastro guiado de dívida + importação de faturas (quando possível).
- Cálculo de taxa efetiva (quando CET não informado, usar estimativa e marcar como suposição).
- Ranking de prioridade (com explicação: juros + risco + prazo).
- Simulador: pagar extra X/mês → meses economizados e juros evitados.
- Alertas: risco de atraso, aumento de taxa, uso de rotativo.

6.6 KPIs e fórmulas

KPI	Fórmula	Interpretação
Dívida/Renda	parcelas_mensais / renda_líquida	Pressão de caixa; acima de ~30% começa a estrangular
Custo anual de juros	juros_pago_12m	Quanto você paga para o passado
Tempo até zero	meses até quitação (simulado)	Ritmo do plano
Juros evitados	juros_base - juros_plano	Retorno direto do esforço

6.7 Catálogo de regras (v1)

- DEBT_INVENTORY_MISSING: detectar sinais de dívida (pagamento mínimo, juros, fatura) sem cadastro → recomendar inventário.

- DEBT_TO_INCOME_HIGH: dívida/renda acima do limiar → plano de contenção e renegociação.
- DEBT_ROTATING_CREDIT: uso de rotativo → alerta crítico + plano (cortar gastos + pagar fatura cheia).
- DEBT_PRIORITY_AVALANCHE: selecionar dívida com maior taxa efetiva como alvo.
- DEBT_EXTRA_PAYMENT: sobra mensal detectada → sugerir aporte extra e mostrar juros evitados.

6.8 UX patterns

- Tela 'Dívidas': ranking por prioridade + botão 'simular pagamento extra'.
- Visual: curva de saldo ao longo do tempo (baseline vs plano).
- Alertas de alto risco (rotativo/atraso) aparecem no topo do app.
- Checklist de renegociação (documentos + perguntas-chave) quando aplicável.

6.9 Agente do capítulo (AgentDebt)

- Missão: reduzir juros e risco de atraso com plano simples.
- Saída: 1 dívida alvo + 3 ações (corte, extra, renegociação) + impacto (meses e juros evitados).
- Guardrails: declarar suposições quando CET desconhecido; não recomendar crédito novo sem justificar.

6.10 Testes essenciais

- Unit: simulador de amortização e juros; ranking de prioridade.
- Integration: faturas/parcelas alimentam corretamente parcelas_mensais e juros.
- E2E: usuário usa rotativo → alerta crítico → plano gerado → acompanhamento mensal reduz saldo.

Definition of Done - Capítulo 6
1) inventário de dívidas; 2) simulador de pagamento extra; 3) regras DEBT_* auditáveis; 4) UI com ranking e curva; 5) alertas rotativo/atraso; 6) testes unit+E2E.

Apêndice C (continuação) - Schemas JSON + DSL de Condições

Nesta versão adicionamos schemas base para Alert, Consent e AgentAction e propomos uma DSL simples para condições (evita 'strings soltas').

C.4 alert.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/alert.schema.json", "title": "Alert", "type": "object",
"required": ["id", "user_id", "level", "title", "message", "source", "created_at"],
"properties": { "id": {"type": "string"}, "user_id": {"type": "string"}, "level":
{"type": "string", "enum": ["info", "attention", "critical"]}, "title": {"type":
"string"}, "message": {"type": "string"}, "source": {"type": "object", "required":
["type"], "properties": { "type": {"type": "string", "enum": ["rule", "system",
"agent"]}, "rule_id": {"type": "string"}, "agent_id": {"type": "string"} }}, "links":
{"type": "array", "items": {"type": "string"}}, "status": {"type": "string", "enum":
["open", "dismissed", "snoozed"]}, "created_at": {"type": "string", "format":
"date-time"} } }
```

C.5 consent.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/consent.schema.json", "title": "Consent", "type":
"object", "required": ["id", "user_id", "scope", "granted", "created_at"],
"properties": { "id": {"type": "string"}, "user_id": {"type": "string"}, "scope":
{"type": "string", "description": "e.g., open_finance_read, export_data,
agent_autopilot"}, "granted": {"type": "boolean"}, "expires_at": {"type": "string",
"format": "date-time"}, "context": {"type": "object", "additionalProperties": true},
"created_at": {"type": "string", "format": "date-time"} } }
```

C.6 agent_action.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/agent_action.schema.json", "title": "AgentAction",
"type": "object", "required": ["id", "user_id", "agent_id", "action", "created_at"],
"properties": { "id": {"type": "string"}, "user_id": {"type": "string"}, "agent_id":
{"type": "string"}, "action": {"type": "string", "description": "e.g.,
propose_budget_cut, propose_debt_plan"}, "inputs_ref": {"type": "object",
"additionalProperties": true}, "outputs_ref": {"type": "object",
"additionalProperties": true}, "status": {"type": "string", "enum": ["proposed",
"accepted", "rejected", "executed"]}, "created_at": {"type": "string", "format":
"date-time"} } }
```

C.7 DSL simples para condições (v1)

Problema: conditions como strings livres são difíceis de validar e perigosas. Proposta: DSL declarativa com operadores limitados, validável por schema e compilável para execução.

Princípios da DSL

• Sem código arbitrário • Operadores limitados • Tipos explícitos • Funções puras • Fácil de testar e auditar.

```
// Exemplo 1: reserva mínima { "all": [ { "<": [ {"var":"kpi.reserve_months"}, 3 ] }, { ">": [ {"var":"kpi.essential_cost_monthly"}, 0 ] } ] } // Exemplo 2: meta subfinanciada com tolerância { "all": [ { "<": [ {"var":"kpi.goal_progress_pct"}, { "-": [ {"var":"kpi.goal_expected_pct"}, {"var":"params.tolerance"} ] } ] }, { ">": [ {"var":"kpi.months_total"}, 0 ] } ] }
```

Operadores suportados (v1)

- Comparação: <, <=, >, >=, ==, !=
- Lógicos: all (AND), any (OR), not
- Aritmética: +, -, *, /
- Acesso: {"var":"kpi.xxx"}, {"var":"params.xxx"}

Próxima evolução (v4+)

Adicionar tipagem (number/string/bool), funções utilitárias (moving_avg, count_in_window) e um validador estático que garante que variáveis existam e tenham tipo correto.

CODEX TÉCNICO

Gestão Financeira Pessoal

v4 adiciona Dimensões 7 e 8 + schemas para carteira (Position/PortfolioTarget) e extensão mínima da DSL para janelas.

Novidades desta versão (v4)
• Capítulo 7 completo (Investimentos: Renda Fixa) • Capítulo 8 completo (Investimentos: Renda Variável) • Apêndice C: Position/PortfolioTarget schemas + extensão DSL (funções de janela).

Índice (v4)

- Capítulo 7 - Investimentos: Renda Fixa
- Capítulo 8 - Investimentos: Renda Variável
- Apêndice C (continuação): schemas JSON (Position, PortfolioTarget) + DSL (funções de janela v1.1)

Capítulo 7 - Investimentos: Renda Fixa

7.1 Definições e escopo

Renda fixa é a família de instrumentos cujo retorno é definido por um indexador/regras contratuais (pré, pós, inflação) e cujo risco principal é: crédito, liquidez e marcação a mercado. Este capítulo define como o sistema classifica, mede e recomenda decisões educacionais sobre renda fixa, sem prescrever produtos específicos.

7.2 Objetivo

- Usar renda fixa com função clara: reserva, curto prazo, âncora defensiva, proteção inflacionária.
- Gerir prazo e liquidez (evitar 'meta curta em papel longo').
- Medir retorno líquido e real com premissas explícitas.

7.3 Decisões-chave

- Indexador: CDI/SELIC (pós), IPCA (inflação), Pré (taxa fixa).
- Prazo e liquidez: D+0/D+1 vs vencimento.
- Risco de crédito: soberano, bancário, corporativo; concentração por emissor/garantia.
- Marcação a mercado: aceitar volatilidade (se carregar até o vencimento) ou exigir liquidez estável.
- Tributação e custos: comparar sempre no líquido.

7.4 Etapas - humano

- Definir função do dinheiro: reserva / curto / médio / longo.
- Escolher liquidez mínima por bucket.
- Evitar concentração em 1 emissor ou 1 vencimento.
- Construir escada de vencimentos (ladder) quando faz sentido.
- Revisar trimestralmente: concentração, prazos e retorno real.

7.5 Etapas - sistema

- Classificar cada posição: indexador, vencimento, liquidez, emissor, risco_credito (se fornecido), tributação.
- Calcular retorno líquido estimado (com premissas de IR e inflação).
- Mapear curva de vencimentos (maturity ladder) e concentração por emissor.
- Detectar mismatch: metas curtas vs ativos ilíquidos/longos.
- Sugerir estrutura: reserva em liquidez + ladder para demais objetivos (educacional).

7.6 KPIs e fórmulas

KPI	Fórmula (simplificada)	Interpretação
Retorno Líquido Estimado	retorno_bruto - imposto - custos	Comparação justa entre alternativas
Retorno Real Estimado	(1+ret_liq)/(1+inflacao)-1	Poder de compra (premissa explícita)
Prazo Médio (WAM)	Σ(valor*anos)/Σ(valor)	Risco de prazo agregado
Concentração por Emissor	max(valor_emissor)/total	Risco de crédito concentrado

7.7 Catálogo de regras (v1)

- FI_LIQUIDITY_FOR_RESERVE: reserva_emergencia alocada em ativo com liquidez > D+1 → alerta + sugestão de correção.
- FI_MATURITY_LADDER_GAP: concentração de vencimentos em 1 janela → sugerir ladder (dividir vencimentos).
- FI_ISSUER_CONCENTRATION: concentração por emissor acima do limite → alerta de risco de crédito.
- FI_REAL_RETURN_NEGATIVE: retorno real estimado < 0 → explicação + alternativas defensivas (sem prescrever produto).
- FI_TAX_AWARE_COMPARE: comparação entre ativos exige sempre bruto→líquido→real (forçar padrão EDU_BRUTO_VS_LIQUIDO).

7.8 UX patterns

- Tela 'Renda Fixa': ladder visual por vencimento (0-6m, 6-12m, 1-2a, 2-5a, 5a+).
- Card por posição: indexador, vencimento, liquidez, emissor, retorno líquido e real (com premissa).
- Alerta de mismatch: meta curta + ativo longo → botão 'ver impacto' (simulador).

7.9 Agente do capítulo (AgentInvestRF)

- Missão: organizar renda fixa por função e reduzir risco invisível (liquidez/prazo/crédito).
- Saída: 3 ajustes máximos (ex.: mover reserva para liquidez; reduzir concentração; criar ladder).
- Guardrails: declarar premissas (inflação, IR); não prometer retorno; não recomendar emissor específico sem dados completos.

7.10 Testes essenciais

- Unit: retorno líquido e real; WAM; concentração por emissor; ladder buckets.
- Integration: posições alimentam corretamente dashboard e alertas de liquidez.
- E2E: usuário marca um bucket como 'reserva' → sistema valida liquidez e dispara correção se necessário.

Definition of Done - Capítulo 7
1) cadastro/classificação de posições RF; 2) KPIs líquido/real e WAM; 3) ladder e concentração; 4) regras FI_* auditáveis; 5) UI RF; 6) testes unit+E2E.

Capítulo 8 - Investimentos: Renda Variável

8.1 Definições e escopo

Renda variável inclui instrumentos cujo preço varia diariamente (ações, ETFs, FIIs etc.). Os riscos principais são volatilidade, drawdown, concentração e comportamento (vender em pânico, comprar por euforia). Este capítulo define disciplina, limites e rebalanceamento.

8.2 Objetivo

- Expor parte do patrimônio a crescimento de longo prazo com limites claros.
- Evitar decisões emocionais (timing de mercado).
- Manter carteira alinhada ao risco e ao horizonte.

8.3 Decisões-chave

- Abordagem: ETFs amplos (simplicidade) vs seleção de ativos (complexidade).
- Percentual máximo de RV por perfil e fase de vida (parâmetro).
- Rebalanceamento: por tempo (mensal/trimestral) e por desvio (ex.: ±5 p.p.).
- Regra de aportes: recorrente (DCA) como padrão.

8.4 Etapas - humano

- Definir % máximo de RV e aceitar volatilidade como preço do retorno.
- Aportar regularmente (independente de humor).
- Evitar concentração em poucos ativos/temas.
- Rebalancear com disciplina (comprar o que caiu, vender o que subiu) dentro de limites.

8.5 Etapas - sistema

- Registrar posições (ticker, quantidade, custo, país/moeda, tipo).
- Calcular alocação atual vs alvo e desvios.
- Detectar concentração (por ativo, setor, país, moeda) quando houver metadados.
- Sugerir rebalanceamento com base em política (tempo + limiar).
- Aplicar fricções comportamentais em eventos de pânico/euforia (Cap. 5).

8.6 KPIs e fórmulas

KPI	Fórmula	Interpretação
Desvio vs Alvo	aloc_atual - aloc_alvo	Gatilho de rebalanceamento
Concentração Top 1/5	Σ top posições / total	Risco idiossincrático
Drawdown (aprox.)	pico_valor - valor_atual / pico_valor	Queda máxima relativa (se histórico)
Giro de Carteira	vendas+compras / total	Sinal de trading emocional

8.7 Catálogo de regras (v1)

- RV_MAX_EXPOSURE: % RV acima do limite do perfil → recomendação de rebalanceamento defensivo.

- RV_CONCENTRATION_HIGH: top 1 ou top 5 acima do limite → alerta de concentração.
- RV_REBALANCE_THRESHOLD: desvio vs alvo > limiar → propor rebalance (via aporte preferencial).
- RV_HIGH_TURNOVER: giro acima do limite → intervenção comportamental (pausa + explicação).
- RV_DCA_REMINDER: mês sem aporte recorrente → lembrar disciplina (se usuário configurou DCA).

8.8 UX patterns

- Tela 'Carteira': alocação atual vs alvo (barras) + 1 CTA: 'rebalancear com aporte'.
- Aviso de concentração: mostrar 'se este ativo cair 40%, impacto no total = X%'.
- Rebalanceamento preferencial por aportes (evitar giro e impostos).
- Fricção de pânico: quando vender em queda → 'pausa de 24h' opcional.

8.9 Agente do capítulo (AgentInvestRV)

- Missão: manter disciplina e limites para crescimento de longo prazo.
- Saída: 1) principais desvios; 2) rebalance sugerido (por aportes); 3) alerta de concentração; 4) uma fricção comportamental se necessário.
- Guardrails: não fazer stock-picking sem dados/consentimento; não prometer retorno; declarar incerteza.

8.10 Testes essenciais

- Unit: cálculo de alocação e desvios; concentração top 1/5; regras de limiar.
- Integration: posições + targets produzem recomendações e logs consistentes.
- E2E: alocação sai do alvo → sistema sugere rebalance por aportes → usuário aceita → alocação converge.

Definition of Done - Capítulo 8
1) posições RV cadastradas; 2) targets e desvios; 3) concentração e regras RV_*; 4) UI carteira com CTA único; 5) fricções comportamentais integradas; 6) testes unit+E2E.

Apêndice C (continuação) - Schemas de Carteira + DSL (v1.1)

Para capítulos 7-9, precisamos de estruturas de posições e alvos. Também estendemos a DSL com funções de janela para regras comportamentais e de aportes.

C.8 position.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/position.schema.json", "title": "Position", "type":
"object", "required": ["id", "user_id", "asset_type", "symbol", "quantity",
"avg_cost", "currency"], "properties": { "id": {"type":"string"}, "user_id":
{"type":"string"}, "asset_type": {"type":"string",
"enum":["fixed_income","stock","etf","reit","fund","crypto","other"]}, "symbol":
{"type":"string"}, "quantity": {"type":"number"}, "avg_cost": {"type":"number"},
"market_price": {"type":"number"}, "currency": {"type":"string"}, "issuer":
{"type":"string"}, "indexer": {"type":"string", "description":"for fixed_income:
CDI/IPCA/PRE"}, "maturity_date": {"type":"string", "format":"date"}, "liquidity":
{"type":"string", "description":"e.g., D0, D1, maturity"}, "tags": {"type":"array",
"items":{"type":"string"}}, "updated_at": {"type":"string", "format":"date-time"} } }
```

C.9 portfolio_target.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/portfolio_target.schema.json", "title":
"PortfolioTarget", "type": "object", "required": ["id", "user_id", "version",
"targets"], "properties": { "id": {"type":"string"}, "user_id": {"type":"string"},
"version": {"type":"string"}, "rebal_policy": {"type":"object", "properties": {
"freq": {"type":"string", "enum":["monthly","quarterly","yearly"]}, "threshold_pp":
{"type":"number"} }}, "targets": { "type":"array", "items":{ "type":"object",
"required":["bucket","asset_type","target_pct"], "properties":{
"bucket":{"type":"string", "description":"e.g., emergency/short/medium/long"},
"asset_type":{"type":"string"}, "target_pct":{"type":"number", "minimum":0,
"maximum":1} } } }, "created_at": {"type":"string", "format":"date-time"} } }
```

C.10 DSL - funções de janela (v1.1)

Para regras como 'pico pós-salário' e 'mês sem aporte', adicionamos funções puras de janela. Elas operam em séries agregadas e retornam valores numéricos/booleanos.

```
// Funções propostas (assinaturas) {"fn":"sum_in_window", "args":[
{"var":"series.spend_total_daily"}, 7 ]} // soma últimos 7 dias
{"fn":"count_in_window", "args":[ {"var":"series.tx_above_limit"}, 30 ]} // contagem
últimos 30 dias {"fn":"moving_avg", "args":[ {"var":"series.spend_total_daily"}, 30 ]}
// média móvel 30 dias {"fn":"exists_in_window", "args":[
{"var":"series.contribution_events"}, 31 ]} // houve aporte no último mês? // Exemplo:
pico 48h após salário (simplificado) { "all":[ {">":[
{"fn":"sum_in_window","args":[{"var":"series.spend_48h_after_salary"},2]},
{"var":"params.spike_threshold"} ]}, {">":[ {"var":"kpi.income_monthly"}, 0 ]} ] } //
Exemplo: mês sem aporte DCA (se configurado) { "all":[ {"==":[
{"var":"params.dca_enabled"}, true ]}, {"==":[
{"fn":"exists_in_window","args":[{"var":"series.dca_contribution_events"},31]}, false
]} ] }
```

Nota

Essas funções assumem que existe uma camada de agregação (series.*). Isso mantém a DSL simples e o motor rápido. A camada de agregação é testável e cacheável.

CODEX TÉCNICO

Gestão Financeira Pessoal

v5 adiciona Dimensões 9 e 10 + dados de preços/performance/câmbio para suportar risco, retorno e rebalanceamento baseado em histórico.

Novidades desta versão (v5)
• Capítulo 9 completo (Carteira & Rebalanceamento avançado) • Capítulo 10 completo (Aposentadoria / Independência Financeira) • Apêndice C: PriceSnapshot/Performance/FxRate schemas.

Índice (v5)

- Capítulo 9 - Carteira & Rebalanceamento avançado
- Capítulo 10 - Aposentadoria / Independência Financeira
- Apêndice C (continuação): schemas JSON (PriceSnapshot, Performance, FxRate)

Capítulo 9 - Carteira & Rebalanceamento avançado

9.1 Definições e escopo

Carteira é o conjunto de posições e seu plano de alocação. Rebalanceamento é o processo de trazer a alocação de volta ao alvo com custo mínimo (impostos, taxas, fricção e risco). Este capítulo integra: targets (Cap. 8), buckets (Cap. 3), liquidez/prazo (Cap. 7) e comportamento (Cap. 5).

9.2 Objetivo

- Manter risco sob controle (não deixar a carteira 'derivar' para o que mais subiu).
- Reduzir decisões reativas (rebalancear por política, não por notícia).
- Minimizar custos: preferir rebalanceamento por aportes e, quando possível, por vendas mínimas.

9.3 Decisões-chave

- Política: rebalance por tempo (mensal/trimestral) + por limiar (ex.: ±5 p.p.).
- Hierarquia de execução: (1) novos aportes → (2) realocação entre buckets → (3) vendas (último recurso).
- Restrições: impostos, custos, liquidez, prazo e regras pessoais (ex.: não vender antes de X meses).
- Métrica de risco: simples (ex.: desvio, concentração) antes de sofisticada (VaR).

9.4 Etapas - humano

- Definir targets por bucket e asset_type (já em PortfolioTarget).
- Escolher política de rebalance (freq + threshold).
- Aportar mensalmente direcionando para os 'abaixo do alvo'.
- Revisar trimestralmente: desvios, concentração, prazo médio e liquidez.

9.5 Etapas - sistema

- Calcular alocação atual por bucket/asset_type e comparar com targets.
- Gerar plano de rebalance por aportes (lista de compras sugeridas por classe).
- Se aporte insuficiente, sugerir realocação entre buckets compatíveis.
- Se ainda insuficiente, simular vendas mínimas com custos/impacto (marcar como 'último recurso').
- Registrar decisão do usuário e atualizar targets quando o usuário deliberadamente muda a estratégia.

9.6 KPIs e fórmulas

KPI	Fórmula	Interpretação
Tracking Error (simples)	Σ |aloc_atual - aloc_alvo|	Quanto a carteira desviou do plano
Custo do Rebalance	imposto + taxas + spread	Penalidade para decidir entre vender/aportar
Risco de Concentração	top5 / total (ou HHI)	Risco idiossincrático crescente
Deriva de Risco	% RV atual - % RV alvo	Sinal de que o risco mudou sem decisão consciente

9.7 Catálogo de regras (v1)

- PORT_REBALANCE_THRESHOLD: desvio vs alvo acima do limiar → gerar plano por aportes.
- PORT_REBALANCE_TIME: chegou a data da política (mensal/trimestral) → revisão com 1 CTA.
- PORT_DRIFT_RISK: deriva de risco (ex.: RV acima do alvo) → alerta + opções de correção.
- PORT_CONCENTRATION: concentração top5 acima do limite → reduzir exposição ou diversificar via aportes.
- PORT_SELL_LAST_RESORT: só propor vendas se (aporte + realocação) não corrigirem em N ciclos.

9.8 UX patterns

- Tela 'Rebalance': 1 visão (alvo vs atual) + 1 ação (rebalance por aporte).
- Explicação sempre em custo/benefício: risco reduzido vs custo (imposto/tempo).
- Estado 'último recurso' para vendas: exige confirmação extra e mostra alternativas conservadoras.

9.9 Agente do capítulo (AgentPortfolio)

- Missão: manter risco alinhado e reduzir custo de correção.
- Saída: plano de rebalance (por aporte) + 2 alertas (risco e concentração) + impacto em KPIs.
- Guardrails: priorizar aportes; declarar custos; evitar giro; integrar fricções do Cap. 5 em decisões de venda.

9.10 Testes essenciais

- Unit: tracking error; desvio vs alvo; custo do rebalance; concentração.
- Integration: PortfolioTarget + Position + PriceSnapshot geram alocação consistente.
- E2E: desvio > limiar → plano por aporte → usuário aceita → tracking error reduz no próximo ciclo.

Definition of Done - Capítulo 9
1) targets e política; 2) alocação atual calculada; 3) plano por aportes; 4) vendas como último recurso; 5) regras PORT_* auditáveis; 6) testes unit+E2E.

Capítulo 10 - Aposentadoria / Independência Financeira

10.1 Definições e escopo

Aposentadoria/Independência Financeira (IF) é a capacidade de sustentar o padrão de vida sem depender da renda do trabalho. No produto, isso vira: meta de patrimônio, taxa de poupança, horizonte, risco e simulações. Importante: resultados dependem de premissas (retorno, inflação, longevidade) e devem ser mostrados como cenários.

10.2 Objetivo

- Dar clareza: 'quanto preciso' e 'quando chego lá' com cenários.
- Manter estratégia simples e robusta (evitar overfitting).
- Conectar hoje → IF: transformar aporte mensal em trajetória de patrimônio e renda futura estimada.

10.3 Decisões-chave

- Método: regra de retirada segura (ex.: 3%–4%/ano) como baseline, com cenários.
- Custos na aposentadoria: essenciais vs discricionários; saúde; dependentes.
- Retorno real esperado: conservador/base/otimista (parâmetros).
- Proteções: reserva, seguros (cap. futuro), diversificação (cap. 9).

10.4 Etapas - humano

- Definir custo de vida desejado em valores reais (hoje).
- Escolher idade/ano alvo e margem de segurança (ex.: +20%).
- Definir taxa de poupança e automatizar aportes.
- Revisar anualmente: mudanças de vida, renda, custos e objetivos.

10.5 Etapas - sistema

- Estimar gasto anual essencial (Cap. 2) e total (Cap. 1).
- Calcular patrimônio alvo via taxa de retirada (ex.: alvo = gasto_anual / swr).
- Simular trajetória de patrimônio com aporte mensal e cenários de retorno real.
- Produzir 'Mapa IF': hoje → marcos (100k, 500k, 1M, etc.) e % do alvo.
- Alertas: contribuição insuficiente, aumento de custo estrutural, risco excessivo perto do prazo.

10.6 KPIs e fórmulas

KPI	Fórmula (simplificada)	Interpretação
Patrimônio Alvo (IF)	gasto_anual_real / swr	Quanto precisa em valores de hoje
% do Alvo	patrimonio_atual / alvo	Progresso rumo à IF
Data IF (cenário)	simulação (aporte + retorno real)	Quando chega no alvo (por cenário)
Margem de Segurança	(patrimonio - alvo) / alvo	Folga (ou falta)

10.7 Catálogo de regras (v1)

- FI_TARGET_CALC: calcular alvo IF com swr e gasto anual (premissas explícitas).

- FI_CONTRIB_GAP: simulação mostra data IF além do desejado → sugerir aumento de aporte ou ajuste de expectativa.
- FI_RISK_NEAR_TARGET: usuário perto do alvo e risco (RV) acima do recomendado → sugerir reduzir volatilidade.
- FI_COST_STRUCTURAL_UP: aumento do gasto essencial por 3 meses → recalcular alvo e alertar.
- FI_SCENARIO_TRANSPARENCY: sempre mostrar 3 cenários (conservador/base/otimista) e declarar premissas.

10.8 UX patterns

- Tela 'Mapa IF': progresso (% do alvo) + 3 datas (cenários) + 1 ação: 'melhorar plano'.
- Mostrar sempre valores reais (ajustados por inflação) e permitir alternar para nominal.
- Narrativa: 'o que muda se você aumentar aporte em R$ X?' (simulação rápida).

10.9 Agente do capítulo (AgentFI)

- Missão: manter o plano de IF realista e robusto.
- Saída: 1) alvo e progresso; 2) cenário de data; 3) 1 alavanca principal (aporte/custo/risco).
- Guardrails: premissas explícitas; não prometer retornos; sempre oferecer alternativa conservadora.

10.10 Testes essenciais

- Unit: cálculo do alvo via swr; simulação determinística simples; cenários.
- Integration: patrimônio atual calculado a partir de Position + Account + Asset.
- E2E: usuário muda aporte → datas IF atualizam → recomendações FI_CONTRIB_GAP mudam coerentemente.

Definition of Done - Capítulo 10
1) cálculo do alvo IF; 2) simulações com 3 cenários; 3) UI mapa IF; 4) regras FI_* auditáveis; 5) integração com carteira e orçamento; 6) testes unit+E2E.

Apêndice C (continuação) - Preços, Performance e Câmbio

Para suportar KPIs de retorno, drawdown e alocação em múltiplas moedas, definimos snapshots de preço, performance agregada e taxas de câmbio.

C.11 price_snapshot.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/price_snapshot.schema.json", "title": "PriceSnapshot",
"type": "object", "required": ["symbol", "date", "price", "currency", "source"],
"properties": { "symbol": {"type":"string"}, "date": {"type":"string",
"format":"date"}, "price": {"type":"number"}, "currency": {"type":"string"}, "source":
{"type":"string"}, "meta": {"type":"object", "additionalProperties": true} } }
```

C.12 performance.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/performance.schema.json", "title": "Performance", "type":
"object", "required": ["id", "user_id", "period_start", "period_end", "currency",
"metrics"], "properties": { "id": {"type":"string"}, "user_id": {"type":"string"},
"period_start": {"type":"string", "format":"date"}, "period_end": {"type":"string",
"format":"date"}, "currency": {"type":"string"}, "metrics": { "type":"object",
"required":["portfolio_value_start","portfolio_value_end"], "properties":{
"portfolio_value_start":{"type":"number"}, "portfolio_value_end":{"type":"number"},
"contributions":{"type":"number"}, "withdrawals":{"type":"number"},
"pnl":{"type":"number"}, "return_pct":{"type":"number"},
"max_drawdown_pct":{"type":"number"} }, "additionalProperties": true }, "created_at":
{"type":"string", "format":"date-time"} } }
```

C.13 fx_rate.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/fx_rate.schema.json", "title": "FxRate", "type":
"object", "required": ["base", "quote", "date", "rate", "source"], "properties": {
"base": {"type":"string", "description":"e.g., USD"}, "quote": {"type":"string",
"description":"e.g., BRL"}, "date": {"type":"string", "format":"date"}, "rate":
{"type":"number"}, "source": {"type":"string"} } }
```

Observação prática

Se você não tiver integração de preços, mantenha market_price opcional e opere com custo médio + aportes. Para uma v1 de produto, isso já sustenta diagnóstico de alocação e disciplina. Preços entram quando você quiser métricas de risco (drawdown) e retorno mark-to-market.

CODEX TÉCNICO

Gestão Financeira Pessoal

v6 adiciona Dimensões 11 e 12 + dados estruturados para impostos e seguros (TaxEvent, Policy, Claim).

Novidades desta versão (v6)
• Capítulo 11 completo (Impostos & otimização fiscal) • Capítulo 12 completo (Seguros & proteção patrimonial) • Apêndice C: TaxEvent/Policy/Claim schemas.

Índice (v6)

- Capítulo 11 - Impostos & otimização fiscal
- Capítulo 12 - Seguros & proteção patrimonial
- Apêndice C (continuação): schemas JSON (TaxEvent, Policy, Claim)

Capítulo 11 - Impostos & otimização fiscal

11.1 Definições e escopo

Este capítulo trata de como o sistema modela e explica impactos tributários sobre renda, investimentos e transações. O objetivo é otimizar decisões dentro da legalidade, sem aconselhamento jurídico/fiscal individual. A lógica é: comparar sempre no líquido, registrar eventos tributáveis e evitar erros comuns (ex.: giro alto com imposto inesperado).

11.2 Objetivo

- Dar visibilidade do que é tributável e do que é isento (por categoria/asset_type).
- Exibir custos fiscais como parte do custo total de uma decisão.
- Reduzir surpresas: previsões simples de imposto por evento e por período.

11.3 Decisões-chave

- Nível de granularidade: por transação (evento) + resumo mensal/anual.
- Premissas: alíquotas parametrizadas (por país/regra) e sempre versionadas.
- Estratégia de cálculo: estimativa conservadora quando dados incompletos (e marcar como suposição).
- Prioridade de UX: imposto como linha (bruto → imposto → líquido → real) em qualquer comparação.

11.4 Etapas - humano

- Cadastrar/confirmar regime e país (quando aplicável).
- Conferir eventos relevantes (venda, resgate, proventos, juros).
- Planejar decisões grandes (vendas/rebalance) considerando imposto e prazo.
- Revisar mensalmente: imposto estimado vs realizado (quando houver).

11.5 Etapas - sistema

- Classificar eventos tributáveis: venda com ganho, rendimentos, juros, proventos, resgates.
- Calcular imposto estimado por evento (com parâmetros e evidência).
- Consolidar por período: total de imposto estimado, base de cálculo, flags de risco.
- Exibir comparativos: sempre bruto→líquido→real (Cap. 4).
- Gerar alertas: giro alto, venda frequente, ausência de dados (custo médio), riscos de inconsistência.

11.6 KPIs e métricas

KPI	Fórmula	Interpretação
Tax Drag (aprox.)	imposto_pago / retorno_bruto	Quanto o imposto reduz o retorno
Imposto estimado (período)	Σ imposto_evento	Previsão de caixa para tributos
Giro tributável	vendas_tributáveis / patrimônio	Risco de custo fiscal por excesso de giro
Qualidade de dados fiscais	% eventos com base completa	Confiança na estimativa

11.7 Catálogo de regras (v1)

- TAX_GROSS_TO_NET_ENFORCE: qualquer comparação de investimento deve mostrar imposto e custos separadamente.
- TAX_EVENT_MISSING_DATA: evento potencialmente tributável sem custo médio/base → solicitar dados e marcar estimativa como baixa confiança.
- TAX_HIGH_TURNOVER_WARNING: giro tributável acima do limiar → alertar custo fiscal + sugerir rebalance por aportes.
- TAX_CASH_NEED_ESTIMATE: imposto estimado do mês acima de X → alertar necessidade de caixa para tributos.
- TAX_ASSUME_CONSERVATIVE: se regime/aliquota desconhecida → aplicar cenário conservador e declarar suposição.

11.8 UX patterns

- Padrão visual: tabela 'Bruto / Imposto / Custos / Líquido / Real'.
- Badge de confiança: alta/média/baixa (dependente de dados completos).
- Resumo mensal: 1 card 'Impostos estimados' + 1 ação 'garantir caixa'.
- Rebalance: mostrar custo fiscal estimado antes de sugerir venda.

11.9 Agente do capítulo (AgentTax)

- Missão: reduzir surpresas fiscais e melhorar decisão no líquido.
- Saída: top 3 eventos com maior impacto fiscal + plano de mitigação (aportes, prazo, evitar giro).
- Guardrails: declarar que é estimativa; não substituir contador; parametrizar país/regime; registrar premissas.

11.10 Testes essenciais

- Unit: imposto por evento, somatório por período, tax drag.
- Integration: TaxEvent gerado a partir de Position/Transaction e aparece em dashboards.
- E2E: venda com ganho → TaxEvent → imposto estimado → alerta de caixa se exceder limiar.

Definition of Done - Capítulo 11
1) modelagem TaxEvent; 2) bruto→líquido→real em comparações; 3) resumo mensal de imposto; 4) regras TAX_* auditáveis; 5) métricas de confiança; 6) testes unit+E2E.

Capítulo 12 - Seguros & proteção patrimonial

12.1 Definições e escopo

Proteção patrimonial aqui é: reduzir impacto financeiro de eventos raros e caros (saúde, vida, incapacidade, patrimônio). O sistema não vende seguro; ele organiza cobertura, gaps, custos e decisões. Inclui: vida, residencial, automóvel, saúde, responsabilidade civil (quando aplicável).

12.2 Objetivo

- Garantir que riscos catastróficos tenham cobertura mínima.
- Evitar pagar caro por cobertura redundante/ineficiente.
- Transformar seguro em algo gerenciável: inventário, renovação, sinistros e gaps.

12.3 Decisões-chave

- Quais riscos são 'catastróficos' para o usuário (depende de dependentes, dívidas, patrimônio).
- Escolha de franquias: custo mensal vs risco de desembolso.
- Cobertura vs capacidade de absorção: o que pode ser auto-segurado (com reserva) e o que não pode.
- Renovação e revisão anual (evitar apólice esquecida).

12.4 Etapas - humano

- Inventariar todas as apólices: tipo, cobertura, prêmio, franquia, vigência, beneficiários.
- Definir prioridades: vida (dependentes), saúde, patrimônio (casa), responsabilidade (se relevante).
- Avaliar gaps: cobertura insuficiente vs risco; redundâncias.
- Revisar anualmente e ao mudar de vida (filhos, casa, renda).
- Registrar sinistros e pós-mortem do evento (lições).

12.5 Etapas - sistema

- Cadastro guiado de apólice (Policy) + importação de documentos (quando possível).
- Cálculo de custo anual e índice de cobertura (aprox.) por risco.
- Detecção de gaps: ausência de seguro essencial ou cobertura abaixo de parâmetro.
- Alertas de renovação (30/15/3 dias) e mudanças relevantes (nascimento de dependentes, compra de imóvel).
- Registro de sinistro (Claim) com valores, prazos e checklist de documentos.

12.6 KPIs e métricas

KPI	Fórmula	Interpretação
Custo anual de seguros	Σ prêmio_anual	Peso do 'custo de proteção'
Cobertura vs Risco (proxy)	cobertura / risco_estimado	Se existe gap (aproximado)
Gaps críticos	contagem de riscos sem cobertura mínima	Prioridades imediatas
Eficiência (proxy)	cobertura / prêmio	Comparar apólices ao longo do tempo

12.7 Catálogo de regras (v1)

- INS_POLICY_INVENTORY_MISSING: sinais de pagamento a seguradora sem Policy cadastrada → solicitar cadastro.
- INS_RENEWAL_DUE: vigência a 30/15/3 dias → alertas graduais + checklist de revisão.
- INS_CRITICAL_GAP: risco catastrófico sem cobertura mínima → alerta crítico + orientar busca de cotação.
- INS_REDUNDANCY: duas apólices para mesmo risco/beneficiário → sugerir revisão por redundância.
- INS_DEDUCTIBLE_TOO_LOW: prêmio alto e franquia muito baixa → simular trade-off (franquia vs prêmio).

12.8 UX patterns

- Tela 'Proteção': matriz risco x cobertura com cores (ok / atenção / crítico).
- Card de apólice: tipo, cobertura, franquia, prêmio, vigência, beneficiários.
- Fluxo de renovação: checklist + comparação de prêmio/cobertura (sem vender).
- Sinistro: timeline de etapas + documentos + status.

12.9 Agente do capítulo (AgentInsurance)

- Missão: proteger contra eventos catastróficos com custo racional.
- Saída: 1) gaps críticos; 2) apólices a revisar; 3) 1 ajuste de franquia/cobertura com trade-offs.
- Guardrails: não recomendar seguradora específica; declarar suposições de risco; priorizar dependentes e dívidas.

12.10 Testes essenciais

- Unit: cálculo custo anual, janelas de renovação, detecção de redundância.
- Integration: Policy gera alertas INS_RENEWAL_DUE; Claim conecta a Policy e atualiza KPIs.
- E2E: nova apólice cadastrada → matriz de proteção atualiza → gaps críticos diminuem.

Definition of Done - Capítulo 12
1) modelagem Policy e Claim; 2) matriz risco x cobertura; 3) alertas de renovação; 4) regras INS_* auditáveis; 5) UX de sinistro; 6) testes unit+E2E.

Apêndice C (continuação) - Schemas: TaxEvent, Policy, Claim

Abaixo estão schemas base para suportar impostos e seguros de forma auditável e extensível. Valores e regras são parametrizáveis e dependem de jurisdição e produto.

C.14 tax_event.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/tax_event.schema.json", "title": "TaxEvent", "type":
"object", "required": ["id", "user_id", "date", "event_type", "asset_type",
"currency", "estimated_tax"], "properties": { "id": {"type":"string"}, "user_id":
{"type":"string"}, "date": {"type":"string", "format":"date"}, "event_type":
{"type":"string",
"enum":["sale_gain","sale_loss","dividend","interest","coupon","redemption","other"]},
"asset_type": {"type":"string"}, "symbol": {"type":"string"}, "quantity":
{"type":"number"}, "proceeds": {"type":"number"}, "cost_basis": {"type":"number"},
"gain": {"type":"number"}, "currency": {"type":"string"}, "jurisdiction":
{"type":"string"}, "rule_version": {"type":"string"}, "estimated_tax":
{"type":"number"}, "confidence": {"type":"number", "minimum":0, "maximum":1},
"assumptions": {"type":"array", "items":{"type":"string"}}, "source_refs":
{"type":"object", "additionalProperties": true}, "created_at": {"type":"string",
"format":"date-time"} } }
```

C.15 policy.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/policy.schema.json", "title": "Policy", "type": "object",
"required": ["id", "user_id", "policy_type", "premium_annual", "start_date",
"end_date"], "properties": { "id": {"type":"string"}, "user_id": {"type":"string"},
"policy_type": {"type":"string",
"enum":["life","health","home","auto","disability","liability","travel","other"]},
"provider": {"type":"string"}, "premium_annual": {"type":"number"}, "deductible":
{"type":"number"}, "coverage_amount": {"type":"number"}, "currency":
{"type":"string"}, "beneficiaries": {"type":"array", "items":{"type":"string"}},
"insured_assets": {"type":"array", "items":{"type":"string"}}, "start_date":
{"type":"string", "format":"date"}, "end_date": {"type":"string", "format":"date"},
"documents": {"type":"array", "items":{"type":"string"}}, "status": {"type":"string",
"enum":["active","expired","cancelled"]}, "created_at": {"type":"string",
"format":"date-time"} } }
```

C.16 claim.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/claim.schema.json", "title": "Claim", "type": "object",
"required": ["id", "user_id", "policy_id", "date", "status"], "properties": { "id":
{"type":"string"}, "user_id": {"type":"string"}, "policy_id": {"type":"string"},
"date": {"type":"string", "format":"date"}, "incident_type": {"type":"string"},
"amount_claimed": {"type":"number"}, "amount_paid": {"type":"number"}, "currency":
{"type":"string"}, "status": {"type":"string",
"enum":["opened","in_review","approved","rejected","paid","closed"]}, "documents":
{"type":"array", "items":{"type":"string"}}, "timeline": {"type":"array", "items":{
"type":"object", "properties":{ "at":{"type":"string","format":"date-time"},
"note":{"type":"string"} } }}, "created_at": {"type":"string", "format":"date-time"} }
}
```

Nota de engenharia

Impostos e seguros são altamente dependentes de jurisdição e contrato. Por isso, o sistema deve versionar regras (rule_version), registrar premissas (assumptions) e expor nível de confiança. A parte 'hard' é menos o cálculo e mais a auditabilidade.

CODEX TÉCNICO

Gestão Financeira Pessoal

v7 adiciona Dimensões 13 e 14: integrações (Open Finance/pipelines) + segurança/privacidade/auditoria para agentes autônomos operarem com guardrails.

Novidades desta versão (v7)
• Capítulo 13 completo (Open Finance & Integrações / pipelines de dados) • Capítulo 14 completo (Privacidade, consentimento, segurança e auditoria) • Apêndice C: DataConnection/IngestionJob/AuditLog schemas.

Índice (v7)

- Capítulo 13 - Open Finance & Integrações / pipelines de dados
- Capítulo 14 - Privacidade, consentimento, segurança e auditoria
- Apêndice C (continuação): schemas JSON (DataConnection, IngestionJob, AuditLog, Permission)

Capítulo 13 - Open Finance & Integrações / pipelines de dados

13.1 Definições e escopo

Integrações trazem dados externos (contas, cartões, investimentos) para reduzir atrito e elevar qualidade. O desafio real é pipeline: consentimento, coleta, normalização, deduplicação, reconciliação, versionamento e observabilidade. Este capítulo define um modelo de integração seguro e auditável, adequado para agentes.

13.2 Objetivo

- Automatizar entrada de dados com alta confiabilidade e baixo custo de suporte.
- Manter integridade (não duplicar, não perder, não distorcer).
- Expor sempre nível de confiança e origem (source-of-truth) por dado.

13.3 Princípios inegociáveis

- Consent-first: sem coleta sem consentimento explícito e escopo claro.
- Imutabilidade de eventos: transações importadas são eventos; correções geram novos eventos (não reescrever histórico).
- Proveniência: toda linha deve guardar origem (instituição, conta, id externo, horário).
- Idempotência: reprocessar não pode duplicar.
- Observabilidade: logs, métricas e alertas de falhas.

13.4 Arquitetura de referência (v1)

- Conectores: DataConnection (instituição + escopos + status).
- Ingestão: IngestionJob (fetch incremental, paginação, retries).
- Normalização: mapear para modelo canônico (Account/Transaction/Position).
- Deduplicação: chave composta (connection_id + external_id + posted_at + amount).
- Reconciliação: comparar saldo vs soma de eventos; criar divergências (recon tasks).
- Camada de qualidade: score e flags (missing_category, missing_cost_basis, etc.).

13.5 Etapas - humano

- Conectar instituição e conceder escopos mínimos necessários.
- Validar primeira sincronização (ex.: últimas 90 dias) e corrigir categorias/padrões.
- Definir regra de conciliação: como tratar pendentes, estornos, duplicatas.
- Revisar semanalmente: divergências e itens 'sem confiança'.

13.6 Etapas - sistema

- Onboarding de conexão com Consent (escopos e validade).
- Sync incremental com checkpoints (cursor, since, page_token).
- Transformação determinística: externo → canônico (com mapeamento versionado).
- Reprocessamento seguro (idempotente) e DLQ (dead-letter queue) para erros.
- Regras de reconciliação e criação de tasks para usuário quando necessário.

13.7 KPIs operacionais

KPI	Definição	Alvo (v1)

Sync Success Rate	% jobs concluídos sem erro crítico	>= 99%
Lag de sincronização	tempo do último evento importado	< 6h (batch)
Taxa de duplicata	% eventos descartados por dedup	< 0.5%
Recon Divergence Rate	% contas com divergência saldo	< 2%
Coverage de categorização	% transações categorizadas	> 90% (com regras)

13.8 Catálogo de regras (v1)

- PIPE_SYNC_FAILED: job falhou N vezes → alertar + abrir task (reautenticar / aguardar).
- PIPE_STALE_SYNC: lag acima do limiar → alertar e priorizar sync.
- PIPE_RECON_DIVERGENCE: divergência saldo acima do tolerável → criar recon task + bloquear recomendações de alta confiança.
- PIPE_DUPLICATE_SPIKE: duplicatas acima do normal → possível bug de cursor → abrir incidente.
- PIPE_DATA_QUALITY_LOW: score de qualidade baixo (muitos campos faltando) → pedir correções e reduzir automações.

13.9 UX patterns

- Tela 'Conexões': status (ok/atenção/erro) + último sync + botão 'reconectar'.
- Badge 'origem': manual vs importado; mostrar instituição e data.
- Fila de reconciliação: 3 tarefas máximas por semana para não cansar.
- Modo 'somente leitura' quando qualidade é baixa: evita recomendações erradas.

13.10 Agente do capítulo (AgentPipelines)

- Missão: manter o sistema confiável. Sem dados confiáveis, não há diagnóstico.
- Saída: 1) saúde do pipeline; 2) top 3 inconsistências; 3) plano de correção e reprocessamento.
- Guardrails: não executar ações destrutivas; qualquer correção deve ser idempotente e auditada.

13.11 Testes essenciais

- Unit: mapeamento canônico; dedup key; recon checks.
- Integration: reprocessar job não duplica; retries com backoff; DLQ.
- E2E: conexão → sync → transações aparecem → recon task criada quando divergência.

Definition of Done - Capítulo 13
1) DataConnection + Consent; 2) IngestionJob com checkpoints; 3) normalização + dedup; 4) recon tasks; 5) observabilidade (logs/metrics); 6) testes unit+E2E.

Capítulo 14 - Privacidade, consentimento, segurança e auditoria

14.1 Definições e escopo

Esta dimensão define como o produto protege dados sensíveis e como agentes autônomos podem operar com segurança. Aqui entram: consentimento, escopos, mínimo privilégio, trilha de auditoria, criptografia, segregação, RLS e prevenção de ações perigosas.

14.2 Objetivo

- Garantir que o usuário tenha controle: o que foi coletado, por quê, e por quanto tempo.
- Reduzir superfície de ataque: mínimos dados, mínimos privilégios.
- Tornar o sistema auditável: toda ação relevante deve ser rastreável.

14.3 Modelo de consentimento

- Consent é um objeto versionado com escopo (ex.: open_finance_read, export_data, agent_autopilot).
- Consent pode expirar e ser revogado; ações devem verificar Consent em tempo de execução.
- Mudanças de escopo exigem recaptura de consentimento (não “upgrade silencioso”).

14.4 Modelo de autorização (RBAC/ABAC)

- RBAC: papéis básicos (user, admin interno, support_readonly).
- ABAC: políticas por atributo (user_id, consent_scope, resource_owner, purpose).
- Regra central: toda consulta e mutação deve ser filtrada por user_id (RLS).

14.5 Guardrails para agentes autônomos

- Ações do agente são propostas primeiro (status=proposed). Execução automática só com consentimento 'agent_autopilot' e lista de ações permitidas.
- Ações perigosas (ex.: excluir dados, alterar targets, executar venda) exigem confirmação explícita e registro em AuditLog.
- O agente nunca deve operar com credenciais de usuário em texto; usa tokens curtos e rotacionáveis.

14.6 Segurança de dados

- Criptografia em trânsito (TLS) e em repouso (provider-managed + campos sensíveis opcionalmente).
- Separação de ambientes (dev/stg/prod) e segregação de chaves.
- Minimização: guardar somente o necessário; PII sensível com retenção curta.
- Backups testados e política de restauração.

14.7 Auditoria

- AuditLog registra: quem (user/agent/system), o que (action), onde (resource), quando, antes/depois (diff) e por quê (purpose).
- A auditoria deve ser imutável (append-only) e pesquisável.
- Eventos críticos geram Alert nível 'critical'.

14.8 KPIs de segurança e conformidade (v1)

KPI	Definição	Uso
Privileged Actions Rate	% ações 'dangerous' / total	Detectar abuso/erro de automação
Consent Coverage	% operações sensíveis com consent válido	Sinaliza falhas de checagem
RLS Coverage	% tabelas com RLS e policies mínimas	Controle de acesso por dado
Audit Completeness	% ações críticas com AuditLog completo	Auditabilidade real

14.9 Catálogo de regras (v1)

- SEC_CONSENT_REQUIRED: tentativa de ação sensível sem consent → bloquear + solicitar consent.
- SEC_CONSENT_EXPIRED: consent expirado → degradar para modo manual e alertar.
- SEC_DANGEROUS_ACTION_CONFIRM: ações 'dangerous' exigem confirmação e registro de reason.
- SEC_AUDIT_MISSING: ação crítica sem AuditLog → incidente (bug) e travar rotas até corrigir.
- SEC_RLS_VIOLATION: tentativa de acessar dados de outro user_id → alertar + bloquear + registrar.

14.10 UX patterns

- Centro de privacidade: listar consents, escopos, validade e botão de revogar.
- Tela 'Atividades': trilha de auditoria legível (o que mudou e por quê).
- Permissões do agente: toggle por tipo de ação (ex.: 'categorizar', 'sugerir', 'executar').
- Modo 'read-only' quando consent expirou ou pipeline está inconsistente.

14.11 Agente do capítulo (AgentSecurity)

- Missão: garantir guardrails e auditabilidade.
- Saída: 1) consents expirando; 2) ações perigosas recentes; 3) gaps de RLS/auditoria.
- Guardrails: nunca contornar consent; preferir bloquear do que assumir.

14.12 Testes essenciais

- Unit: verificação de consent por escopo; classificação de ação (safe vs dangerous).
- Integration: RLS policies; audit append-only; log de diffs.
- E2E: ação sensível sem consent → bloqueada + prompt de consent; com consent → executa + audit.

Definition of Done - Capítulo 14
1) consent + escopos; 2) RLS e políticas mínimas; 3) AuditLog append-only; 4) guardrails do agente; 5) centro de privacidade; 6) testes unit+E2E.

Apêndice C (continuação) - Schemas: DataConnection, IngestionJob, AuditLog, Permission

Schemas base para integrações e segurança. Eles amarram pipeline + consent + auditoria. Ajuste conforme o stack (Supabase, etc.).

C.17 data_connection.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/data_connection.schema.json", "title": "DataConnection",
"type": "object", "required":
["id","user_id","provider","status","scopes","created_at"], "properties": {
"id":{"type":"string"}, "user_id":{"type":"string"},
"provider":{"type":"string","description":"institution or aggregator name"},
"status":{"type":"string","enum":["active","error","revoked","paused"]},
"scopes":{"type":"array","items":{"type":"string"}}, "consent_id":{"type":"string"},
"external_ref":{"type":"object","additionalProperties":true},
"last_sync_at":{"type":"string","format":"date-time"}, "cursor":{"type":"string"},
"created_at":{"type":"string","format":"date-time"} } }
```

C.18 ingestion_job.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/ingestion_job.schema.json", "title": "IngestionJob",
"type": "object", "required": ["id","connection_id","status","started_at"],
"properties": { "id":{"type":"string"}, "connection_id":{"type":"string"},
"status":{"type":"string","enum":["queued","running","success","failed","partial"]},
"since":{"type":"string","format":"date-time"},
"until":{"type":"string","format":"date-time"}, "cursor_before":{"type":"string"},
"cursor_after":{"type":"string"}, "items_fetched":{"type":"integer"},
"items_written":{"type":"integer"}, "duplicates_dropped":{"type":"integer"},
"errors":{"type":"array","items":{"type":"string"}},
"started_at":{"type":"string","format":"date-time"},
"finished_at":{"type":"string","format":"date-time"} } }
```

C.19 audit_log.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/audit_log.schema.json", "title": "AuditLog", "type":
"object", "required":
["id","user_id","actor_type","action","resource_type","resource_id","at"],
"properties": { "id":{"type":"string"}, "user_id":{"type":"string"},
"actor_type":{"type":"string","enum":["user","agent","system","support"]},
"actor_id":{"type":"string"}, "action":{"type":"string"},
"risk_level":{"type":"string","enum":["safe","sensitive","dangerous"]},
"purpose":{"type":"string"}, "resource_type":{"type":"string"},
"resource_id":{"type":"string"},
"before":{"type":"object","additionalProperties":true},
"after":{"type":"object","additionalProperties":true},
"diff":{"type":"object","additionalProperties":true}, "request_id":{"type":"string"},
"ip_hash":{"type":"string"}, "at":{"type":"string","format":"date-time"} } }
```

C.20 permission.schema.json (v1)

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://example.com/schemas/permission.schema.json", "title": "Permission", "type":
"object", "required": ["id","user_id","subject","action","resource","effect"],
"properties": { "id":{"type":"string"}, "user_id":{"type":"string"},
"subject":{"type":"string","description":"agent_id or role"},
```

```
"action":{"type":"string","description":"e.g., categorize_tx, export_data,
execute_plan"}, "resource":{"type":"string","description":"e.g., transactions, reports,
portfolio"}, "effect":{"type":"string","enum":["allow","deny"]},
"conditions":{"type":"object","additionalProperties":true},
"created_at":{"type":"string","format":"date-time"} } }
```

Fechamento técnico

Com v7, o Codex passa a suportar: dados confiáveis (pipeline) + decisões seguras (consent/RLS/audit). Isso é a base para 'autonomous agents' sem virar um risco operacional.

CODEX TÉCNICO

Gestão Financeira Pessoal

v8 fecha o ciclo operacional: Dimensões 15 e 16 (Operações do produto + Plataforma de agentes). Isso transforma o Codex em manual de execução, não só blueprint.

Novidades desta versão (v8)
• Capítulo 15 completo (Operações do produto: métricas, qualidade, incidentes, SLOs) • Capítulo 16 completo (Plataforma de agentes: roteamento, ferramentas, memória, avaliação, red-teaming) • Apêndice C: Incident/SLO/EvalRun/PromptPackage schemas.

Índice (v8)

- Capítulo 15 - Operações do produto: métricas, qualidade, incidentes, SLOs
- Capítulo 16 - Plataforma de agentes: roteamento, ferramentas, memória, avaliação, red-teaming
- Apêndice C (continuação): schemas JSON (Incident, SLO, EvalRun, PromptPackage)

Capítulo 15 - Operações do produto: métricas, qualidade, incidentes, SLOs

15.1 Definições e escopo

Operações do produto (ProdOps) define como você garante confiabilidade no dia a dia: instrumentação, métricas, qualidade de dados, incidentes, suporte e SLOs. Para finanças pessoais, o maior risco é o produto gerar confiança com dados errados. Portanto, qualidade e observabilidade viram feature.

15.2 Objetivo

- Controlar qualidade do dado e confiabilidade do diagnóstico.
- Detectar e resolver incidentes rápido (minimizar dano reputacional).
- Construir ciclo de melhoria contínua: bugs → métricas → correção → prevenção.

15.3 Taxonomia de qualidade

- Freshness: dado atualizado (lag baixo).
- Completeness: campos críticos preenchidos (categoria, conta, moeda, custo).
- Accuracy: reconciliação e validações sem divergências.
- Consistency: regras invariantes (saldo não bate? duplicata? valores negativos inconsistentes?).
- Explainability: usuário entende a origem e as premissas.

15.4 Métricas (North Star + operacionais)

- North Star (exemplo): % de usuários com 'visão confiável' (score de qualidade >= X) + uso de insights.
- Aquisição/ativação: % que cadastra 1 conta + 20 transações em 7 dias.
- Retenção: WAU/MAU; retorno a telas-chave (orçamento, carteira, alertas).
- Qualidade: % transações categorizadas; recon divergence; duplicatas; lag de sync.
- Segurança: ações perigosas; falhas de consent; auditoria incompleta.

15.5 SLOs (exemplos para v1)

SLO	SLI	Meta	Observação
Disponibilidade	reqs 2xx / total	>= 99.9% mensal	exclui manutenção programada
Freshness pipeline	lag p95	<= 6h	para integrações batch
Reconciliação	% contas sem divergência	>= 98%	tolerância por tipo
Agente: segurança	% ações sensíveis com consent	100%	hard fail

15.6 Incidentes: severidade e playbooks

- Sev1: dados errados para muitos usuários (duplicação massiva, saldo invertido) → congelar automações + banner + correção + post-mortem.
- Sev2: degradação relevante (sync falhando em 1 provedor) → rota alternativa + comunicação no status.
- Sev3: bug isolado/UX quebrado → hotfix quando possível.
- Playbook padrão: detectar → mitigar → comunicar → corrigir → prevenir (action items).

15.7 Guardrails operacionais

- Modo degradado automático: se qualidade < limiar, o produto reduz recomendações e bloqueia automações.
- Feature flags: ligar/desligar integrações/insights por cohort.
- Kill switch de agente: desliga execução automática globalmente.
- Data backfill controlado: reprocessar com idempotência e limites.

15.8 Testes e releases

- Gate de release: checklist (RLS ok, consent checks ok, audit ok, SLO dashboards ok).
- Canary release: 1% → 10% → 100% com métricas de saúde.
- E2E crítico: cadastro, importação, orçamento, carteira, alertas, agente em modo 'propor'.

Definition of Done - Capítulo 15
1) dashboards de qualidade e pipeline; 2) SLOs definidos e monitorados; 3) incident playbooks; 4) kill switch e modo degradado; 5) gates de release; 6) post-mortem template.

Capítulo 16 - Plataforma de agentes: roteamento, ferramentas, memória, avaliação, red-teaming

16.1 Definições e escopo

A plataforma de agentes é a camada que transforma 'um modelo' em 'um sistema'. Ela decide: qual agente atua, quais ferramentas usa, como registra ações, como aprende sem vazar privacidade, e como você mede qualidade. Aqui o Codex vira motor para múltiplos produtos.

16.2 Objetivo

- Garantir respostas úteis, consistentes e seguras.
- Controlar custo (tokens, latência) e risco (ações perigosas).
- Permitir evolução: prompts versionados, testes A/B, avaliações automáticas.

16.3 Componentes

- Router: classifica intenção (orçamento, dívida, investimentos, impostos, seguro, pipeline, segurança).
- Agents: especialistas por dimensão (AgentBudget, AgentDebt, AgentInvestRF/RV, AgentPortfolio, AgentTax, AgentInsurance, AgentPipelines, AgentSecurity).
- Tools: calculadoras, import/export, fetch dados, simulações, gerador de relatórios.
- Memory: preferências do usuário + decisões (nunca dados sensíveis desnecessários).
- Action ledger: AgentAction + AuditLog para tudo que muda estado.

16.4 Roteamento (policy)

- 1) Detectar intenção + risco (safe/sensitive/dangerous).
- 2) Selecionar agente especialista; se risco alto, forçar modo 'propor'.
- 3) Requerer dados mínimos (ex.: portfolio_target para rebalance).
- 4) Aplicar templates de resposta e guardrails do capítulo 14.

16.5 Memória e privacidade

- Salvar somente: preferências de metas, limiares, estilo de notificações, tolerância a risco.
- Não salvar: números exatos de conta/saldo, PII, detalhes de transações (exceto se usuário exigir e houver base legal).
- Versionar decisões importantes (targets, swr, limites) e permitir rollback.

16.6 Avaliação (Evals)

- Conjunto de cenários fixos (golden set): orçamento, dívida, rebalance, imposto, seguro, pipeline falho, consent expirado.
- Métricas: precisão numérica, completude, clareza, segurança (não violar consent), custo (tokens), latência.
- Rodar evals a cada release; bloquear deploy se segurança falhar.

16.7 Red-teaming (agentes)

- Testar prompts maliciosos: induzir venda, apagar dados, vazar PII, ignorar consent.
- Testar erros 'inocentes': dados faltantes, duplicatas, recon divergente, moeda errada.
- Checklist: o agente bloqueia? pede consent? reduz confiança? registra audit?

16.8 Guardrails: política de ação

- Níveis: sugerir → propor plano → executar (só com autopilot + allowlist).
- Toda execução cria: AgentAction(status=executed) + AuditLog + Alert se risco>=sensitive.
- Kill switch global + por usuário + por provedor.

16.9 Testes essenciais

- Unit: roteamento; classificação de risco; validação de consent.
- Integration: ferramentas + ledger; execução cria audit e updates.
- E2E: user pede rebalance → router seleciona AgentPortfolio → plano proposto → confirmação → execução + audit.

Definition of Done - Capítulo 16
1) router + risk levels; 2) agents especialistas; 3) tools integradas; 4) ledger/audit; 5) evals automatizadas; 6) red-team checklist + kill switch.

Apêndice C (continuação) - Schemas: Incident, SLO, EvalRun, PromptPackage

Schemas para operar o produto e a plataforma de agentes como sistema (monitorável, versionado e testável).

C.21 incident.schema.json (v1)

```
{ "$schema":"https://json-schema.org/draft/2020-12/schema",
"$id":"https://example.com/schemas/incident.schema.json", "title":"Incident",
"type":"object", "required":["id","severity","title","status","opened_at"],
"properties":{ "id":{"type":"string"},
"severity":{"type":"string","enum":["sev1","sev2","sev3"]}, "title":{"type":"string"},
"status":{"type":"string","enum":["open","mitigating","resolved"]},
"impact":{"type":"string"}, "root_cause":{"type":"string"},
"mitigation":{"type":"string"}, "owner":{"type":"string"},
"opened_at":{"type":"string","format":"date-time"},
"resolved_at":{"type":"string","format":"date-time"},
"action_items":{"type":"array","items":{"type":"string"}},
"links":{"type":"array","items":{"type":"string"}} } }
```

C.22 slo.schema.json (v1)

```
{ "$schema":"https://json-schema.org/draft/2020-12/schema",
"$id":"https://example.com/schemas/slo.schema.json", "title":"SLO", "type":"object",
"required":["id","name","sli","target","window"], "properties":{
"id":{"type":"string"}, "name":{"type":"string"}, "sli":{"type":"string"},
"target":{"type":"number"}, "window":{"type":"string","description":"e.g., 30d"},
"alerting":{"type":"object","additionalProperties":true},
"created_at":{"type":"string","format":"date-time"} } }
```

C.23 eval_run.schema.json (v1)

```
{ "$schema":"https://json-schema.org/draft/2020-12/schema",
"$id":"https://example.com/schemas/eval_run.schema.json", "title":"EvalRun",
"type":"object", "required":["id","suite","model","started_at","results"],
"properties":{ "id":{"type":"string"}, "suite":{"type":"string"},
"model":{"type":"string"}, "prompt_package_id":{"type":"string"},
"started_at":{"type":"string","format":"date-time"},
"finished_at":{"type":"string","format":"date-time"},
"results":{"type":"array","items":{ "type":"object", "properties":{
"case_id":{"type":"string"}, "pass":{"type":"boolean"},
"scores":{"type":"object","additionalProperties":true}, "notes":{"type":"string"} } }}
} }
```

C.24 prompt_package.schema.json (v1)

```
{ "$schema":"https://json-schema.org/draft/2020-12/schema",
"$id":"https://example.com/schemas/prompt_package.schema.json",
"title":"PromptPackage", "type":"object", "required":["id","version","agents"],
"properties":{ "id":{"type":"string"}, "version":{"type":"string"},
"agents":{"type":"array","items":{ "type":"object",
"required":["name","system_prompt"], "properties":{ "name":{"type":"string"},
"system_prompt":{"type":"string"}, "tools":{"type":"array","items":{"type":"string"}},
"guardrails":{"type":"array","items":{"type":"string"}} } }},
"created_at":{"type":"string","format":"date-time"} } }
```

Fechamento do Codex (v1)

Com v8, você tem as 16 dimensões + operação + plataforma de agentes. O próximo passo natural é consolidar tudo em um único PDF mestre (vFinal), com índice geral e cross-references entre capítulos.